AdvR 16 – Debugging & Performance

Debugging

Debugging Strategy

> *Finding your bug is a process of confirming the many things that you believe are true — until you find one which is not true.* — *Norm Matloff*

Four-step approach:

1. **Google!** — search the error message (remove variable names/values)
2. **Make it repeatable** — minimal reproducible example, simplify data
3. **Figure out where** — use `traceback()`, `browser()`, systematic experiments
4. **Fix it and test it** — write a test case to prevent regression

`traceback()` and Call Stacks

`traceback()` shows the sequence of calls leading to the error:

```
f <- function(a) g(a)
g <- function(b) h(b)
h <- function(c) i(c)
i <- function(d) {
  if (!is.numeric(d)) stop("`d` must be numeric")
  d + 10
}
f("a")
#> Error: `d` must be numeric
traceback()
#> 4: i(c) at #1
#> 3: h(b) at #1
#> 2: g(a) at #1
#> 1: f("a")
```

Read **bottom** → **top**: `f()` → `g()` → `h()` → `i()` → error. Use `rlang::last_trace()` for tree-structured output.

Interactive Debugger: `browser()`

Insert `browser()` to **pause execution** and explore interactively:

```
g <- function(b) {
  browser()    # execution pauses here
  h(b)
}
```

Commands inside the debugger:

Command	Action
`n` (Next)	Execute next step
`s` (Step into)	Step into the next function call
`f` (Finish)	Finish current loop or function
`c` (Continue)	Resume normal execution
`Q` (Quit)	Stop debugging, return to global

Conditional: `if (b < 0) browser()`.

Alternatives and Non-interactive Debugging

Alternatives to `browser()`:

- **Breakpoints** — click left of line number in RStudio
- `options(error = recover)` — choose any frame to inspect
- `debug(f)` / `debugonce(f)` — auto-pause on function entry

Non-interactive debugging (batch jobs, pipelines):

- `dump.frames(to.file = TRUE)` → load + `debugger()` later
- **Print debugging** — insert `cat()` / `print()` statements
- Unexpected warnings → `options(warn = 2)` (converts to errors)
- Unexpected messages → `rlang::with_abort(f(), "message")`
- R crashes → bug in compiled (C/C++) code

Measuring Performance

Profiling with `profvis`

Profile code to find bottlenecks — don't guess:

```
library(profvis)
profvis({
  f <- function() { pause(0.1); g(); h() }
  g <- function() { pause(0.1); h() }
  h <- function() pause(0.1)
  f()
})
```

profvis output has two panes:

- **Top**: source code + time/memory bars per line
- **Bottom**: **flame graph** — full call stack over time

`<GC>` in the flame graph = garbage collector running → sign of many short-lived objects (e.g., growing a vector in a loop).

Microbenchmarking: `bench::mark()`

Compare small code snippets precisely:

```
x <- runif(100)
bench::mark(
  sqrt(x),
  x ^ 0.5
)[c("expression", "min", "median", "itr/sec")]
```

```
#> # A tibble: 2 x 4
#>   expression      min   median `itr/sec`
#>   <bch:expr> <bch:tm> <bch:tm>     <dbl>
#> 1 sqrt(x)    279.98ns 310.13ns  1456185.
#> 2 x^0.5        1.36us   1.42us   692588.
#> ... [output truncated]
```

Auto-checks all expressions return the **same value** (set `check = FALSE` to override).

Time	Calls per second
1 ms	1,000
1 μs	1,000,000
1 ns	1,000,000,000

Exercises (Measuring)

1. Which is fastest? Predict, then benchmark:

```
x <- runif(100)
sqrt(x);  x ^ 0.5;  x ^ (1/2);  exp(log(x) / 2)
```

2. How do `system.time()` estimates compare to `bench::mark()`? Why might they differ?

Solutions (Measuring)

1.

```
x <- runif(100)
bench::mark(
  sqrt(x), x ^ 0.5,
  x ^ (1/2), exp(log(x) / 2)
)[c("expression", "min", "median", "itr/sec")]
```

```
#> # A tibble: 4 x 4
#>   expression         min   median `itr/sec`
#>   <bch:expr>    <bch:tm> <bch:tm>     <dbl>
#> 1 sqrt(x)       279.86ns 339.93ns  1399991.
#> 2 x^0.5           1.38us   1.44us   548242.
#> ... [output truncated]
```

`sqrt()` is fastest (dedicated C function). `exp(log(x)/2)` is slowest (two function calls).

2. `system.time()` has ~ms precision; `bench::mark()` uses a high-precision timer (~ns). `system.time()` also includes process overhead and can't isolate individual expressions.

Improving Performance

Strategy: Code Organisation

1. Wrap each approach in a **function**
2. Generate a **representative test case** (big enough to matter, small enough to iterate)
3. Benchmark with `bench::mark()` — auto-checks correctness

```
x <- runif(1e5)
mean1 <- function(x) mean(x)
mean2 <- function(x) sum(x) / length(x)
bench::mark(mean1(x), mean2(x)
  )[c("expression", "min", "median")]
```

```
#> # A tibble: 2 x 3
#>   expression      min   median
#>   <bch:expr> <bch:tm> <bch:tm>
#> ... [output truncated]
```

`sum(x)/length(x)` is faster — `mean()` makes two passes for numerical accuracy.

Do Less Work

Use **specialised** functions tailored to your input:

- `rowSums()` / `colMeans()` instead of `apply()`
- `vapply()` instead of `sapply()` (pre-specifies output)
- `any(x == 10)` instead of `10 %in% x`
- `readr::read_csv()` instead of `read.csv()`

Skip **method dispatch** in tight loops:

```
x <- runif(1e2)
bench::mark(
  mean(x), mean.default(x)
)[c("expression", "min", "median", "itr/sec")]
```

```
#> # A tibble: 2 x 4
#>   expression           min   median `itr/sec`
#>   <bch:expr>      <bch:tm> <bch:tm>     <dbl>
#> ... [output truncated]
```

Trade-off: `mean.default()` is faster but fails if `x` isn't numeric.

Vectorisation = whole-object approach; loops run in C, not R.

```
x <- matrix(runif(1e5), nrow = 1000)
bench::mark(
  apply(x, 1, sum), rowSums(x)
)[c("expression", "min", "median", "itr/sec")]
```

```
#> # A tibble: 2 x 4
#>   expression            min   median `itr/sec`
#>   <bch:expr>       <bch:tm> <bch:tm>     <dbl>
#> ... [output truncated]
```

Key vectorised functions: `rowSums()`, `colMeans()`, `cumsum()`, `diff()`, `pmin()`, `pmax()`, `crossprod()`.

Vectorised subsetting is also fast: `x[is.na(x)] <- 0` replaces all NAs in one step.

Avoid Copies

Growing objects in loops is $O(n^2)$ — each iteration copies everything:

```
collapse <- function(xs) {
  out <- ""
  for (x in xs) out <- paste0(out, x)
  out
}
s <- replicate(100, paste(sample(letters, 50,
  replace = TRUE), collapse = ""))
bench::mark(
  loop = collapse(s),
  vectorised = paste(s, collapse = ""),
  check = FALSE
)[c("expression", "min", "median")]
```

```
#> # A tibble: 2 x 3
#>   expression      min   median
#>   <bch:expr> <bch:tm> <bch:tm>
#> ... [output truncated]
```

Rule: pre-allocate output or use a vectorised function.

Case Study: Faster t-test

1000 experiments × 50 individuals. Step 1 — **do less work**:

```r
m <- 1000; n <- 50
X <- matrix(rnorm(m * n, 10, 3), nrow = m)
grp <- rep(1:2, each = n / 2)

my_t <- function(x, grp) {
  t_stat <- function(x) {
    m <- mean(x); n <- length(x)
    list(m = m, n = n,
      var = sum((x - m)^2) / (n - 1))
  }
  g1 <- t_stat(x[grp == 1])
  g2 <- t_stat(x[grp == 2])
  (g1$m - g2$m) /
    sqrt(g1$var/g1$n + g2$var/g2$n)
}
```

Strips formatting, p-values, etc. from `t.test()` — ~6x faster.

Case Study: Vectorised t-test

Step 2 — **vectorise**: `mean()` → `rowMeans()`, `sum()` → `rowSums()`:

```
rowtstat <- function(X, grp) {
  t_stat <- function(X) {
    m <- rowMeans(X); n <- ncol(X)
    var <- rowSums((X - m)^2) / (n - 1)
    list(m = m, n = n, var = var)
  }
  g1 <- t_stat(X[, grp == 1])
  g2 <- t_stat(X[, grp == 2])
  se <- sqrt(g1$var/g1$n + g2$var/g2$n)
  (g1$m - g2$m) / se
}
bench::mark(
  loop = purrr::map_dbl(1:m, ~my_t(X[.,], grp)),
  vectorised = rowtstat(X, grp)
)[c("expression", "min", "median")]
```

```
#> # A tibble: 2 x 3
#>   expression      min   median
#>   <bch:expr> <bch:tm> <bch:tm>
#> ... [output truncated]
```

Exercises (Improving)

1. Compare `apply(x, 1, sum)` vs `rowSums(x)` for a 10000 $\times$ 100 matrix. How large is the speedup?
2. What's the difference between `rowSums()` and `.rowSums()`?

Solutions (Improving)

1.

```
x <- matrix(runif(1e6), nrow = 10000)
bench::mark(
  apply(x, 1, sum), rowSums(x)
)[c("expression", "min", "median", "itr/sec")]
```

```
#> # A tibble: 2 x 4
#>   expression            min   median `itr/sec`
#>   <bch:expr>       <bch:tm> <bch:tm>     <dbl>
#> ... [output truncated]
```

`rowSums()` is ~10–20× faster: implemented in C, avoids R-level loop.

2. `.rowSums(x, m, n)` is the **internal** function — skips dimension checks and dispatching. Slightly faster but requires you to pass `nrow` and `ncol` explicitly. Use `rowSums()` unless profiling shows it's a bottleneck.